Algoritmi 2

I seguenti appunti sono a cura di ***Catalin Ceban*** *e si basano sul corso di "Algoritmi 2" erogato nell'A.A. 2026 dal professore Paul Joseph Wollan per il C.d.L Informatica (33503)*

13 ottobre 2026

Indice

1 Teoria dei Grafi **2**

1.1 Il problema dei ponti di Konigsberg 3

1.2 Rappresentazione fisica dei grafi 5

1.2.1 Matrici di adiacenza 6

1.2.2 Liste di adiacenza 6

1.3 Passeggiate e cammini 8

1.4 Algoritmo di ricerca di un ciclo all'interno di un grafo 9

1.5 Visite di un grafo 13

1.5.1 DFS - Depth-First-Search 13

1.6 Generici problemi su grafi 20

1.6.1 Problema di bicolorazione di un grafo 22

1.7 Lavoro su grafi orientati 24

1.7.1 Ricerca di un ordine topologico 26

1.8 Comportamento nei grafi non diretti 34

1

Teoria dei Grafi

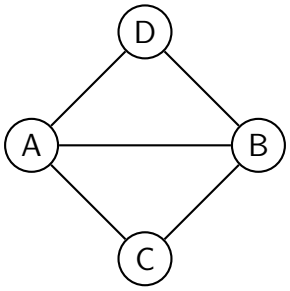
Definizione 1.0.1: Grafo

Un **grafo** è una coppia $G = (V, E)$, dove:

- V rappresenta l'insieme dei **vertici** (o nodi).
- E rappresenta l'insieme degli **archi** (o lati). Matematicamente parlando, un arco è a sua volta una coppia di valori (v_1, v_2), dove v_1, v_2 sono de vertici del grafo.

Per comodità di notazione, nel corso del manuale denoteremo con $n = |V|$ il numero dei vertici e con $m = |E|$ il numero degli archi.

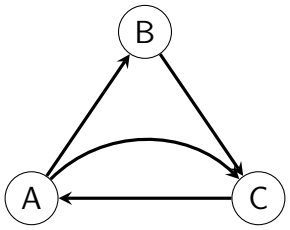
A titolo esemplificativo, consideriamo la seguente rappresentazione grafica di un grafo:



Riguardo agli archi, potremmo avere anche un orientamento.

Definizione 1.0.2: Grafo diretto (o orientato)

Un grafo si dice **diretto** o **orientato** se ciascuno dei suoi archi ha un orientamento, cioè punta specificatamente verso un dato nodo.



È importante notare che possiamo avere anche due archi sugli stessi nodi ma con orientamento diverso (come mostrato nel collegamento bidirezionale).

Definizione 1.0.3: Arco incidente e vertici adiacenti

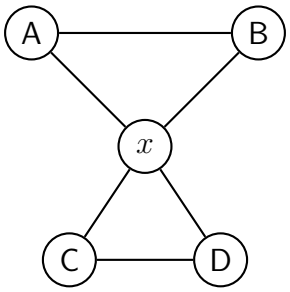
Sia f un arco che collega due vertici x e y, formalmente rappresentabile come $x \longleftarrow f \longrightarrow y$. Diciamo che:

- L'arco f è **incidente** ai vertici x e y.
- Il vertice x è **adiacente** al vertice y.

Definizione 1.0.4: Grado di un vertice

Il **grado** di un vertice x, denotato con $\deg(x)$, è dato dal numero di archi incidenti in x.

Ad esempio, considerando un nodo centrale collegato a quattro vertici periferici:



Per il nodo centrale x contrassegnato, avremo chiaramente $\deg(x) = 4$, poiché vi incidono esattamente quattro archi **distinti**.

1.1 Il problema dei ponti di Konigsberg

Il problema classico da cui nasce la teoria dei grafi è il famoso problema dei ponti di Konigsberg:

Ci si chiede: *è possibile attraversare tutti i ponti una sola volta e tornare al punto di partenza?* Come prima cosa, quando si vuole tradurre un problema in un problema di

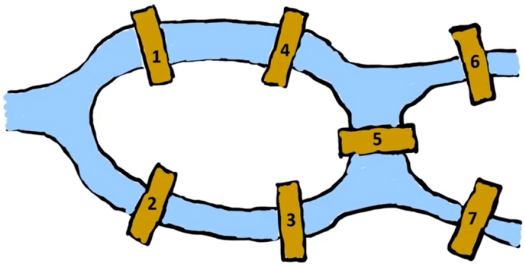


Figura 1.1: Rappresentazione grafica del problema dei ponti di Konigsberg.

teoria dei grafi, è buona cosa cercare di dare una rappresentazione grafica il più accurata possibile del problema. Essendo questo un problema di piccole dimensioni, possiamo ottenere la seguente struttura:

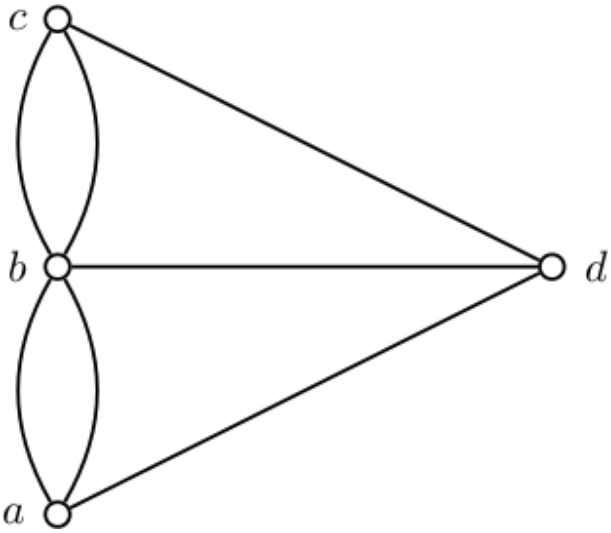


Figura 1.2: Rappresentazione a grafo del problema dei ponti.

Si invita il lettore a provare manualmente se esiste tale sequenza di attraversamenti, ma la risposta alla domanda rimane **negativa**. Prima di giustificarne la motivazione, dobbiamo fornire delle importantissime definizioni riguardanti i grafi.

Definizione 1.1.1: Passeggiata in un grafo

Una **passeggiata** (o **walk**) in un grafo è una sequenza $v_0, e_1, v_1, e_2, v_2, \ldots, v_{k-1}, e_k, v_k$ che alterna vertici e archi. Formalmente, può essere anche indicata con $W = (v_1, v_2, \ldots, v_k)$, dunque omettendo gli archi ma indicando solo in ordine di apparizione i vertici.

Definizione 1.1.2: Passeggiata chiusa

Una passeggiata si dice **chiusa** se $v_0 = v_k$, cioè il punto di inizio coincide con il punto di arrivo.

Possiamo adesso giustificare, più o meno, la risposta al problema dei ponti di Konigsberg: qualsiasi grafo con almeno un nodo x con grado dispari **non** ammette una passeggiata chiusa che utilizza tutti gli archi una sola volta.

E se invece tutti i nodi avessero grado pari? Interessante, domanda, ma **non è detto**.

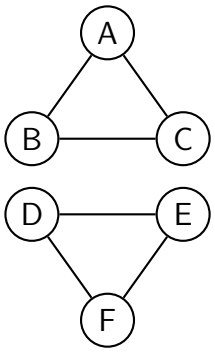


Figura 1.3: Grafo non connesso formato da due componenti triangolari disgiunte. Tutti i vertici hanno $\deg(v) = 2$, ma il grafo complessivo non è connesso e dunque non è possibile effettuare una passeggiata chiusa al suo interno.

Definizione 1.1.3: Grafo connesso

Un grafo si dice **connesso** se per ogni coppia di vertici esiste una passeggiata da x a y. In poche parole, un grafo si dice connesso se ogni coppia di vertici scelta è collegata in qualche modo.

Teorema 1.1.1: Caratterizzazione di Eulero

Esiste una passeggiata chiusa che contiene ogni arco una sola volta se e solo se:

1. Ogni vertice ha **grado pari**.
2. Il grafo è **connesso**.

1.2 Rappresentazione fisica dei grafi

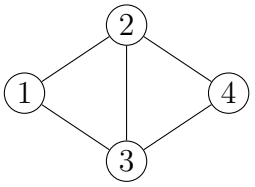
La rappresentazione informatica dei grafi in memoria è fondamentale per determinare l'efficienza degli algoritmi che operano su di essi. Esistono principalmente due modalità standard per rappresentare un grafo: le **matrici di adiacenza** e le **liste di adiacenza**.

1.2.1 Matrici di adiacenza

Si stabilisce una matrice $n \times n$ dove $n = \#V$, e dove:

$$m_{i,j} = \begin{cases} 1 & \text{se } i \text{ è adiacente a } j \\ 0 & \text{altrimenti} \end{cases}$$

Esempio:



$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

Se il grafo non è diretto, la matrice sarà simmetrica rispetto alla diagonale.

Vediamo di seguito il **costo computazionale** di alcune delle operazioni elementari che possono essere effettuate sui grafi:

- **Visita del grafo e memorizzazione**: $\boldsymbol{\Theta(n^2)}$, in quanto viene memorizzato come una matrice $n \times n$.
- Per verificare se **i è adiacente a j**: $\boldsymbol{\Theta(1)}$, in quanto è sufficiente controllare il valore di $M[i][j]$: se esso corrisponde ad 1 allora i due nodi sono adiacenti, altrimenti no.
- **Verificare il grado di un dato vertice i**: $\boldsymbol{\Theta(n)}$, in quanto occorre scorrere la riga i-esima della matrice e contare quanti 1 ci sono.

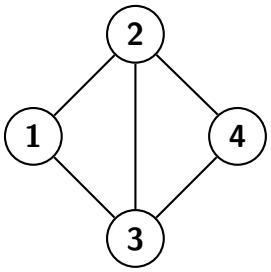
1.2.2 Liste di adiacenza

Una lista di adiacenza è composta da:

- Un array in cui ogni elemento è un nodo del grafo.
- Per ogni nodo del grafo, dunque per ogni elemento dell'array, viene associato anche un puntatore `neigbh` che punta ad una lista concatenata contenente tutti i nodi adiacenti al nodo corrente.

Costo computazionale delle operazioni elementari:

- Per visitare e memorizzare: $\mathcal{O}\left(n + \sum_v \deg(v)\right) = \mathcal{O}(n + 2m) = \mathcal{O}(n + m)$.
  Spieghiamo il perché di questo costo computazionale:



$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \begin{array}{l} \longrightarrow [2,3] \\ \longrightarrow [1,3,4] \\ \longrightarrow [1,2,4] \\ \longrightarrow [2,3] \end{array}$$

1. Sicuramente, per memorizzare una lista di adiacenza abbiamo bisogno innanzitutto di un array che tenga traccia di tutti i nodi. Ecco da dove viene il $\mathcal{O}(n)$.
2. Inoltre, per ogni nodo dobbiamo memorizzare anche tutti i nodi a lui adiacenti. Il numero di nodi adiacenti corrisponde al **grado** del singolo nodo, ossia al numero di archi incidenti al nodo. Sappiamo che il numero di archi è quantificabile con m, ma in questo caso stiamo contando ogni arco ben due volte (vedendo anche l'esempio, nel nodo 1 stiamo dicendo che è adiacente a 2, e nel nodo due stiamo dicendo che è adiacente ad 1). Dovremo dunque contare due volte ogni arco, ottenendo una quantità pari a $2m$. Il costo computazionale diventa dunque nell'ordine di $n+2m$, ma sapendo che le costanti moltiplicative sono indifferenti asintoticamente, tale valore diventa semplicemente $O(n+m)$.

- Per verificare se i è adiacente a j: $\mathcal{O}(\deg(i))$. Questo in quanto come vediamo la lista di adiacenza relativa ad ogni nodo ha un numero di elementi pari al grado di quel nodo. Dunque per verificare se due nodi sono adiacenti o meno è necessario scorrere la lista di adiacenza di quel nodo e cercare il nodo adiacente interessato.

Per quanto riguarda la memorizzazione e la visita completa di un grafo memorizzato mediante liste di adiacenza, si potrebbe fare un appunto riguardo ai grafi diretti. In tal caso, a meno che ogni arco non sia bi-direzionale, verrebbe memorizzata letteralmente **la metà** dei nodi memorizzati in un grafo non orientato.

Osservazione 1.2.1

Supponiamo di avere un problema che ci richieda frequentemente quali nodi puntano verso un dato nodo u. Se mantenessimo un'implementazione mediante liste di adiacenza come quella vista in figura, per trovare tutti i nodi collegati ad u dovremmo scorrere per ogni nodo nella lista le sue liste di adiacenza, ottenendo un costo computazionale di $\mathcal{O}(n+m)$, come se fosse una normale visita.

Se invece decidessimo di modificare leggermente la struttura della lista di adiacenza,

aggiungendo per ogni nodo un puntatore ai suoi nodi entranti e uscenti, il costo computazionale di tale operazione si ridurrebbe ad un misero $\mathcal{O}(\deg(u))$, in quando dovremo scorrere solo la lista dei nodi entranti (il cui numero corrisponde appunto al grado del nodo). Nota bene che questa implementazione non appesantisce così tanto il costo di memorizzazione della struttura dati, in quanto per ogni nodo u si aggiunge $\mathcal{O}(\deg(u))$ di costo computazionale, portando il totale ad $\mathcal{O}(n + 3m = \mathcal{O}(n + m))$.

1.3 Passeggiate e cammini

Definizione 1.3.1: Cammino

Un **cammino** nel grafo è una passeggiata in cui non viene ripetuto alcun vertice.

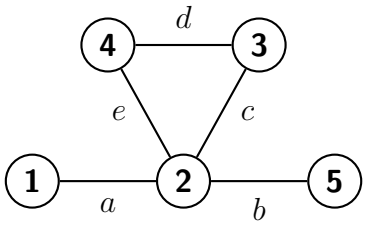


Figura 1.4: Una passeggiata potrebbe essere: 1 *a* 2 *c* 3 *d* 4 *e* 2 *b* 5.
Una camminata da 1 a 5 potrebbe invece essere 1 *a* 2 *b* 5.

Corollario 1.3.1

Esiste una passeggiata da x a y $\iff$ esiste un cammino da x a y.

Dimostrazione 1.3.1

Chiaramente, se esiste un cammino allora sicuramente esiste anche una passeggiata, dato che ogni cammino è un particolare tipo di passeggiata. Abbiamo così dimostrato la prima implicazione ($\Rightarrow$).

Viceversa, supponiamo di avere una passeggiata $W = (v_0, v_1, \ldots, v_K)$. Abbiamo due possibilità:

1. Se tutti i vertici sono distinti, allora la passeggiata è già un cammino.
2. Se invece abbiamo almeno un indice ripetuto, cioè esistono i, j tali che $v_i = v_j$ con $i < j$.

Questo significa che all'interno della passeggiata è presente un sottociclo:

$$W_1 = (v_i, v_{i+1}, \ldots, v_{j-1}, v_j)$$

ma sapendo che $v_i = v_j$, la struttura si chiude. Possiamo eliminare W_1 dalla passeggiata iniziale mantenendo però il nodo in cui parte la diramazione (in questo caso v_i). La passeggiata iniziale può essere dunque riscritta come:

$$W = (v_1, \dots, v_i, \underbrace{v_{i+1}, \dots, v_{j-1}, v_j}_{W_1}, v_{j+1}, \dots, v_K)$$

Eliminando W_1 otteniamo un cammino più corto, e reiterando il procedimento rimuoviamo tutti i cicli ottenendo infine un cammino semplice.

Definizione 1.3.2: Ciclo

Un **ciclo** è un grafo connesso in cui **ogni vertice ha grado 2**.

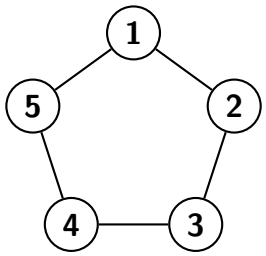
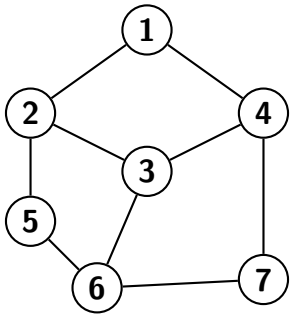


Figura 1.5: Grafo ciclo a 5 vertici (C_5).

1.4 Algoritmo di ricerca di un ciclo all'interno di un grafo

Proponiamo un altro problema classico del lavoro con i grafi: supponiamo che, dato un grafo G con l'ipotesi che **ogni vertice ha grado 2 o superiore**, volessimo trovare un qualsiasi ciclo nel grafo. Partiamo con un esempio grafico di un grafo che contiene un ciclo, e vediamo come procederemmo umanamente.



L'idea di algoritmo chiaramente consiste nel:

1. Scegliere un nodo qualsiasi di partenza.

2. Memorizzarlo all'interno di un apposita struttura dati.

3. Prendere un arco e andare al nodo adiacente. **Importante** è assicurarsi che il prossimo nodo non sia ancora stato visitato. In tal caso, significa che abbiamo fatto un giro e abbiamo trovato un ciclo.

4. Altrimenti, tornare all'inizio.

Messa un'idea di base, procediamo con la scrittura dell'effettivo algoritmo in pseudocodice e in seguito alla sua spiegazione.

```
1   def trovaCiclo(G):
2     L = lista vuota
3     x = un vertice qualsiasi del grafo
4     L.append(x)
5
6     current = x
7     next = x.neigbh[0]
8
9     while next not in L:
10      L.append(next)
11
12      if next.neighbr[0] == current:
13        u = next.neighbr[1]
14      else:
15        u = next.neighbr[0]
16
17      current = next
18      next = u
19
20    while L[0] != next:
21      L.remove(0)
22
23    return L
```

Listing 1.1: Pseudocidice per la ricerca di un ciclo all'interno di un grafo.

Spiegazione algoritmo 1.4.1: Eestrazione di un ciclo da un grafo

1. Definiamo una lista vuota, per tenere traccia dei nodi visitati finora.
2. Prendiamo un qualsiasi vertice dal quale iniziare la nostra ricerca.
3. Aggiungiamo il nodo scelto all'interno della lista.
4. Prendiamo come prossimo nodo il primo della lista degli adiacenti al nodo corrente.
5. Eseguiamo un ciclo che va avanti finché il next che abbiamo preso non è un nodo già visitato: in tal caso significa che abbiamo intrapreso una strada circolare e siamo tornati indietro.
6. All'interno del ciclo aggiungiamo alla lista `L` dei nodi visitati anche `next`.
7. A questo punto dobbiamo scegliere il nuovo `next`, ma dobbiamo assicurarci che non sia il nodo da cui siamo partiti: ecco perché controlliamo che `next.neighbr[0]` sia diverso dal nodo su cui siamo adesso, in modo da non poter tornare *subito* indietro.
8. Aggiorniamo i puntatori dei nodi per la prossima iterazione.
9. Una volta usciti da questo ciclo, significa che abbiamo intrapreso la strada circolare di cui si parlava prima, e che dunque abbiamo trovato un ciclo. Significa inoltre che ora il puntatore `next` punterà al nodo da cui inizia la diramazione del ciclo, e dunque sarà il primo di questo. Ecco perché, nel ciclo finale, controlliamo che `L[0] != next`, perché `next` è il primo nodo del ciclo, e dunque deve essere anche il primo della lista che dovremo restituire.

Bisogna fare un appunto su questo algoritmo: ora come ora, per come l'abbiamo scritto, il suo costo computazionale è un $\mathcal{O}(n^2)$, tutto per colpa del controllo di testa del primo ciclo while. La condizione del ciclo è un `next not in L`, che richiede tempo $\mathcal{O}(n)$ in un array disordinato. Dato che il ciclo while potrebbe poi richiedere fino ad un massimo di n iterazioni, una per ogni nodo del grafo, il costo computazionale sale al tempo quadratico. Possiamo ovviamente ottimizzare questo codice, presentando la seguente versione:

```
1    def trovaCiclo(G):
2      L = lista vuota
3      vis = [0] * n
4
```

```
5       x = un vertice qualsiasi del grafo
6       L.append(x)
7       vis[x] = 1
8
9       current = x
10      next = x.neigbh[0]
11
12      while vis[next] == 0:
13        L.append(next)
14        vis[next] = 1
15
16        if next.neighbr[0] == current:
17          u = next.neighbr[1]
18        else:
19          u = next.neighbr[0]
20
21        current = next
22        next = u
23
24      while L[0] != next:
25        L.remove(0)
26
27    return L
```

Listing 1.2: Ricerca di un ciclo in un grafo ottimizzata

Spiegazione algoritmo 1.4.2: Ricerca di un ciclo in un grafo

Andremo a commentare solo le nuove scelte implementative: Si è scelto di aggiungere un array `vis` che verrà usato per tenere traccia degli elementi visitati. Ciò viene fatto stabilendo una biezione tra l'identificativo del nodo e il suo indice all'interno dell'array `vis`. Così facendo il controllo in testa al ciclo while diventa semplicemente un controllo accedendo ad un elemento dell'array tramite indice, dunque eseguibile in tempo costante. L'aggiunta di un array `vis` che ha una grandezza massima dell'ordine di del numero di nodi n aumenta il costo spaziale da $\Theta(n)$ a $\Theta(n+n) = \Theta(n)$, dunque rimane nello stesso ordine asintotico.

Possiamo ora discutere riguardo al costo computazionale dell'algoritmo, per entrambe le versioni:

- Versione inefficiente:
  - **Costo computazionale:** $\mathcal{O}(n^2)$
  - **Costo spaziale:** $\Theta(n)$
- Versione efficiente:
  - **Costo computazionale:** $\Theta(n)$
  - **Costo spaziale:** $\Theta(n)$

1.5 Visite di un grafo

Prima di discutere i modi in cui si possono effettuare le visite su un grafo, diamo alcune definizioni importanti.

Definizione 1.5.1: Nodo raggiungibile

Un nodo y si dice **raggiungibile** da x se $\exists$ un cammino da x a y.

Definizione 1.5.2: Grafo connesso

Un grafo G si dice **connesso** se $\forall x, y \in V(G)$, $\exists$ un cammino da x a y.

Le visite nascono per rispondere alla problematica classica: *Esiste un cammino da x a y?*

Il metodo più sicuro consiste nel **visitare**, a partire dal nodo x, uno ad uno tutti i nodi del grafo. Ovviamente, se durante tale visita si incontra il nodo y, la risposta alla domanda sarà affermativa.

Ciò si può fare principalmente in due modi:

DFS (*Depth-First-Search*, Visita in profondità): Funziona come l'esplorazione di un labirinto: si sceglie un percorso e si avanza finché non si trova un vicolo cieco. A quel punto si torna indietro fino al primo incrocio in cui si sono lasciate strade inesplorate e si prova un'altra via.

BFS (*Breadth-First-Search*, Visita in ampiezza): Preso un nodo iniziale, si visitano prima tutti i suoi vicini diretti, poi i vicini dei vicini, e così via.

1.5.1 DFS - Depth-First-Search

Vediamo per prima la visita DFS.

In base alla brevissima e introduttiva descrizione dell'algoritmo fornita poco fa, possiamo delinearne un'idea algoritmica di base:

1. Partiamo da un nodo x e lo segniamo come visitato.
2. Scegliamo una strada inesplorata verso cui procedere.
3. Segniamo il nodo scelto come visitato e ripetiamo la visita ricorsivamente o iterativamente.

La modalità di backtracking verrà implementata mediante una pila (*stack*), che lavora secondo il principio LIFO (*Last In, First Out*). Dunque, quando dobbiamo tornare indietro, basta effettuare un'operazione di `pop()` per tornare al nodo precedentemente esaminato.

Vediamo una prima implementazione iterativa dell'algoritmo:

```
DFS(G, x):
  vis = [0] * n
  S = uno stack
  S.push(x)
  vis[x] = 1

  while S.size() != 0:
    y = S.top()
    if esiste un vicino z di y che non è stato visitato (
       vis[z] == 0):
      S.push(z)
      vis[z] = 1
    else:
      S.pop()

  return vis
```

Listing 1.3: Implementazione iterativa della DFS

Spiegazione algoritmo 1.5.1: Analisi dettagliata dell'algoritmo DFS

Analizziamo riga per riga il funzionamento dell'implementazione:

- `vis = [0] * n`: Inizializziamo l'array per tenere traccia dei nodi visitati.
- `S = uno stack`: Inizializziamo lo stack per permettere il backtracking.

- `S.push(x)` e `vis[x] = 1`: Aggiungiamo il nodo di partenza allo stack e lo segniamo come visto.
- `while S.size() != 0`: Iteriamo finché lo stack non rimane vuoto, il che vorrà dire che ogni nodo raggiungibile è stato esaminato.
- `y = S.top()`: Prendiamo il nodo in cima allo stack, che rappresenta il nodo corrente.
- **Controllo del vicino**: Se la condizione di entrata è rispettata (esiste un vicino z tale che `vis[z] == 0`), vuol dire che possiamo continuare per un'altra strada, cioè quella verso z. Di conseguenza effettuiamo il `push` di z e lo marchiamo come visitato.
- **Gestione del vicolo cieco (*else*)**: Altrimenti, significa che abbiamo raggiunto un "vicolo cieco" e che dobbiamo tornare indietro. Nel pratico, questo si traduce nel rimuovere l'elemento in cima alla pila tramite `S.pop()`, così che alla prossima iterazione i controlli vengano fatti sul nodo da cui siamo arrivati e sui suoi rimanenti vicini.

Seppur si tratti di un'implementazione molto descrittiva dell'algoritmo, analizziamo il controllo condizionale: dobbiamo prendere il nodo y, controllarne i vicini e verificare che almeno per un vicino z valga la condizione `vis[z] == 0`.

Questo controllo corrisponde a una ricerca sequenziale che, come ben sappiamo, richiede tempo $O(n)$ nel caso peggiore per singolo nodo, se non opportunamente ottimizzata.

È poi facile verificare come il ciclo `while` si ripeta tante volte quanti sono i nodi del grafo, dunque $\Theta(n)$.

Combinando queste due osservazioni, otteniamo che il costo di questa versione è $O(n^2)$, che risulta non essere particolarmente ottimale.

Proponiamo di seguito una versione ottimizzata dell'algoritmo.

```
1  def DFS_ottimizzato(G, x):
2      vis = [0] * len(G)
3      vis[x] = 1
4
5      s = Stack()
6      s.push(x)
7
```

```
8         while s.size() != 0:
9           current = s.pop()
10          for v in current.neighbor: # (2)
11            if vis[v] == 0:
12              vis[v] = 1
13              s.push(v)
14
15        return vis
```

Spiegazione algoritmo 1.5.2: Visita DFS ottimizzata

La differenza rispetto alla versione non ottimizzata risiede in due passaggi chiave:

1. In **(1)** estraiamo il nodo corrente dalla pila. Così facendo lo eliminiamo dalla pila e non dal grafo, evitando modifiche distruttive alla struttura dati; in questo modo il nodo `current` non verrà mai più controllato nelle iterazioni successive.
2. In **(2)** andiamo a verificare ciascun vertice adiacente a quello appena estratto. Se non è ancora stato visitato, lo marchiamo come visitato e lo inseriamo nella pila, così da poter proseguire l'esplorazione a partire da esso.

Questa versione dell'algoritmo presenta la seguente complessità:

- **Costo spaziale**: $\Theta(n)$, dovuto all'array dei nodi visitati e allo spazio occupato dalla pila nel caso peggiore.
- **Costo temporale**: il ciclo `while` viene eseguito n volte, esattamente una volta per ciascun vertice. All'interno del ciclo viene eseguito un lavoro costante $\Theta(1)$ per l'operazione di estrazione (`pop`) e un costo pari a $O(\deg(\text{current}))$ per scorrere l'intera lista di adiacenza del nodo estratto.

Il costo computazionale complessivo del ciclo è quindi descritto dalla seguente sommatoria:

$$\sum_{i=1}^{n} \Big(O(1) + O(\deg(G[i])) \Big) = \sum_{i=1}^{n} 1 + \sum_{i=1}^{n} \deg(G[i]) = \\ = O(n) + O(2m) = O(n + 2m) = O(n + m)$$

Per completezza, essendo i grafi un soprainsieme degli alberi, scriviamo una versione ricorsiva della DFS divisa in una funzione ausiliaria e nella visita effettiva.

```
def dfs(G, x):
  vis = [0] * n
  return dfs_ric(G, x, vis)

def dfs_ric(G, x, vis):
  vis[x] = 1
  for v in x.neighbr:
    if vis[v] == 0:
      dfs_ric(G, v, vis)
```

La visita costa sempre $O(n+m)$. L'unica limitazione è lo stack delle funzioni, che per grafi molto pieni potrebbe andare in *recursion depth*.

Dimostrazione 1.5.1: Correttezza della DFS ricorsiva

Dimostriamo ora che questo algoritmo funziona.
Appuntiamo che se un nodo y è raggiungibile a partire da x, allora esiste un cammino che collega $x \to y$ e dunque vis$[y] = 1$. Se y è raggiungibile da $x \implies \exists W = (v_0, v_1, \dots, v_k)$ cammino che collega x (v_0) e y (v_k).
Per assurdo, assumiamo che vis$[y] = 0$. Siccome vis$[x] = 1 \implies \exists$ un indice i con vis$[v_i] = 1$ e vis$[v_{i+1}] = 0$. Se vis$[v_i] = 1$ allora v_i è stato visitato ed è stato inserito nello stack S. Dato che l'algoritmo è terminato, significa che ad un certo punto v_i è stato rimosso dallo stack.
Contraddizione: se è stato rimosso, vuol dire che tutti gli adiacenti a v_i erano a loro volta stati visitati. Tuttavia prima avevamo detto che vis$[v_{i+1}] = 0$, dunque abbiamo dimostrato per assurdo che l'algoritmo funziona.

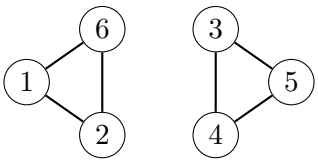
Proponiamo ora un esercizio che ci aiuterà a lavorare con i grafi e con la visita in profondità (DFS).

Esercizio 1.5.1: Calcolo delle componenti connesse tramite DFS

Modificare l'algoritmo di visita DFS al fine di calcolare num_comp, ovvero il numero totale di componenti connesse di un grafo, e popolare un array comp[] con valori appartenenti a $\{1, \dots, \text{num_comp}\}$ tale che:

$$\text{comp}[x] = \text{comp}[y] \iff x \text{ e } y \text{ appartengono alla stessa componente connessa}$$

Esempio. Consideriamo il seguente grafo disconnesso formato da due componenti connesse:



In questo caso il risultato dell'algoritmo sarà:

$$\text{num_comp} = 2, \qquad \text{comp} = [1, 1, 2, 2, 2, 1]$$

dove la posizione i-esima dell'array comp indica la componente di appartenenza del nodo i.

Diamo prima un'idea generale della struttura dell'algoritmo:

1. **Inizializzazione delle strutture dati globali**: Inizializziamo l'array globale comp di dimensione n impostando tutti i valori a 0. Impostiamo inoltre la variabile globale num_comp = 0.

2. **Avvio della DFS**: Selezioniamo il primo nodo non ancora visitato, incrementiamo il contatore (num_comp = num_comp + 1) ed eseguiamo la DFS a partire da esso.

3. **Assegnazione della componente durante la DFS**: All'interno della funzione DFS, per ogni vertice v raggiunto durante la visita, impostiamo:

   $$\text{comp}[v] = \text{num_comp}$$

   L'utilità dell'array `comp` sta anche nel fatto che può benissimo rimpiazzare l'array `vis` che avevamo precedentemente usato per marcare i nodi visitati. Avendolo inizializzato con tutti 0, i nodi non ancora visitati avranno `comp[v] = 0`.

4. **Scorrimento di tutti i vertici**: Terminata la singola DFS, si prosegue scorrendo l'elenco dei vertici del grafo:

   - Se `comp[v] != 0`, il nodo è già stato visitato e possiamo passare al successivo.
   - Altrimenti, incrementiamo `num_comp = num_comp +1` e riavviamo la DFS dal nodo v.

Avendo discusso l'idea dell'algoritmo, possiamo fornirne un esempio di implementazione

```
def trova_comp(G):
  comp = [0] * n
  num_comp = 0
  S = Stack()

  for v in G:
    if comp[v] == 0:
      num_comp += 1
      S.push(v)
      comp[v] = num_comp

      while S.size() != 0:
        current = S.pop()
        for w in current.neighbor:
          if comp[w] == 0:
            comp[w] = num_comp
            S.push(w)

  return comp, num_comp
```

Spiegazione algoritmo 1.5.3: Ricerca delle componenti connesse

- **Inizializzazione**: Viene creato un vettore `comp` di dimensione n inizializzato a 0, che servirà anche a segnalare i nodi visitati, un contatore `num_comp` impostato a 0 e una struttura dati di tipo `Stack`.
- **Identificazione di una nuova componente**: Nel ciclo `for v in G`, l'istruzione `if comp[v] == 0` verifica se il nodo v non è ancora stato visitato. Se questa condizione è vera, significa che ci troviamo su una **nuova componente connessa**, in quanto il nodo corrente non appartiene a nessuna componente precedentemente esplorata.
- **Assegnazione ed esplorazione**:
  1. Si incrementa il contatore delle componenti (`num_comp += 1`).
  2. Si inserisce il nodo sorgente v nello stack `S` e se ne assegna l'indice della componente corrispondente (`comp[v] = num_comp`).

3. Tramite il ciclo `while S.size() != 0`, si procede all'estrazione di ogni nodo (`current = S.pop()`) ed all'analisi dei suoi vicini (`for w in current.neighbor`). Per ciascun vicino non ancora visitato (`comp[w] == 0`), viene registrata l'appartenenza alla componente corrente e il nodo viene inserito nello stack per ulteriori esplorazioni.

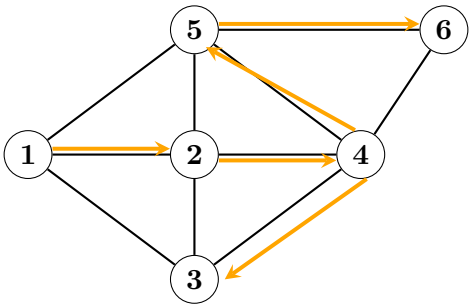
Il costo computazionale dell'algoritmo è lo stesso di una normale DFS, dunque risulta essere $\mathcal{O}(n+m)$. Si potrebbe pensare che la DFS venga eseguita una volta per ogni nodo, ma per come abbiamo segnalato i nodi visitati, l'esplorazione viene eseguita **una sola volta per ogni componente connessa**.

1.6 Generici problemi su grafi

Prima di continuare il discorso sulle visite, vediamo alcuni esempi di operazioni sui grafi diretti (o orientati).

Definizione 1.6.1: Albero di ricerca

Dati due nodi x e y, gli archi che utilizziamo per raggiungere y a partire da x formano il cosiddetto **albero di ricerca**.
Si ricorda che un albero è definito come un **grafo connesso e aciclico**.



Tale albero viene memorizzato mediante il cosiddetto **"vettore dei padri"**, ovvero un array in cui, per ogni indice i, memorizziamo la provenienza dell'esplorazione.

Nell'esempio precedente, il vettore dei padri è rappresentato da:

$$P = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 1, & 1, & 4, & 2, & 4, & 5 \end{bmatrix}$$

in cui, per ogni elemento ad indice i, è specificato da quale nodo proviene i.

Il nodo radice dell'albero si trova identificando quell'elemento in cui $P[i] == i$, cioè il valore del nodo è uguale al suo indice nell'array.

Esercizio 1.6.1: Distanza di un nodo dalla radice

Dato un vettore dei padri P e un vertice x, vogliamo trovare la distanza di x dalla radice dell'albero.

```
def distRad(P, x):
  count = 0
  while P[x] != x:
    x = P[x]
    count += 1
  return count
```

Listing 1.4: Risoluzione iterativa

```
def distRad(P, x):
  if P[x] == x:
    return 0
  return 1 + distRad(P, P[x])
```

Listing 1.5: Risoluzione ricorsiva

Questo algoritmo potrà essere utilizzato in futuro nel corso.

Avendo definito il vettore dei padri, possiamo implementare la visita in profondità (DFS) sfruttando tale struttura.

```
def dfs_ric(G, x, P):
  for v in x.neighbor:
    if P[v] == null:
      P[v] = x
      dfs_ric(G, v, P)

def dfs(G, x):
  P = [null] * n
  P[x] = x
  dfs_ric(G, x, P)
  return P
```

Listing 1.6: Algoritmo DFS con vettore dei padri

1.6.1 Problema di bicolorazione di un grafo

Consideriamo un insieme di n corsi e m studenti. Vogliamo programmare gli esami in due settimane, cioè in modo tale che due esami differenti non possano essere collocati nella stessa settimana.

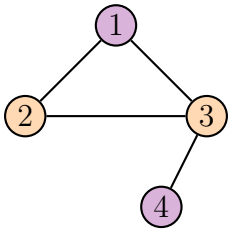
Supponiamo che ciascuno studente debba sostenere esattamente due corsi. Questa assunzione è motivata dal fatto che, se uno studente avesse un numero di corsi superiore a tre, sarebbe impossibile evitare di posizionarne almeno due nella stessa settimana.

E' molto importante che, nella risoluzione di un problema di grafi, sappiamo individuare quale dato nel problema corrisponde ai nodi e quale agli archi.

Nel nostro caso, consideriamo un esempio con 4 corsi e 4 studenti, dove ogni studente è associato a una coppia di corsi da frequentare e superare:

- Corso 1: studenti $\{1, 2\}$
- Corso 2: studenti $\{2, 3\}$
- Corso 3: studenti $\{3, 4\}$
- Corso 4: studenti $\{1, 3\}$

Come possiamo osservare, siccome ogni studente è una coppia di esami, questa corrisponde esattamente alla definizione di arco in un grafo. Dunque il grafo $G = (\text{Corsi}, \text{studenti})$ risultante per questo esempio è rappresentato di seguito:

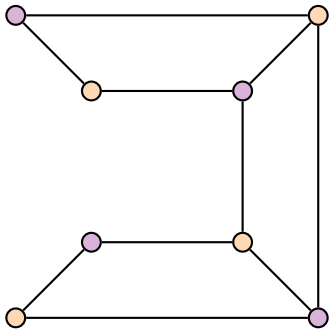


Dove i colori dei nodi indicano la settimana assegnata all'esame:

- Settimana 1
- Settimana 2

Come possiamo notare, nell'esempio fornito non è possibile soddisfare il requisito desiderato senza violare i vincoli, poiché i vertici 1, 2 e 3 formano un triangolo (un ciclo di lunghezza dispari).

Il problema della programmazione degli esami in due settimane si riconduce al classico **problema di bi-colorazione dei grafi**, ovvero determinare se è possibile assegnare uno di due colori (le due settimane) a ciascun vertice in modo tale che ogni arco congiunga due vertici di colore diverso.



Proposizione 1.6.1: Impossibilità della bi-colorazione

Il problema di bi-colorazione **non ha soluzione** per i grafi che contengono **cicli di lunghezza dispari**.

L'intuizione dietro questa proprietà è che, percorrendo un ciclo di lunghezza dispari alternando i colori, dovendo assegnare colori diversi a vertici adiacenti, si arriverà inevitabilmente a un punto in cui due vertici adiacenti richiederanno lo stesso colore, rendendo impossibile una colorazione valida.

Quando poniamo un nodo ad un gruppo (ad esempio 1), allora tutti i suoi adiacenti dovranno appartenere all'altro gruppo (2) e viceversa.

Vediamo l'effettivo codice che, preso in input un grafo, restituisce un Array `part` rappresentante la partizione. Se l'Array restituito risulta essere vuoto, significa che tale grafo non era bi-colorabile.

```
    def dfs_part(G, x):
      part = [0] * n
      part[x] = 1
      s = stack()
      s.push(x)

      while s.size() != 0:
        y = s.top()
        for v in y.neighbr:
          if part[v] == 0:
            part[v] = 3 - part[y]
```

```
12              s.push(v)
13            else:
14              if part[v] == part[y]:
15                return []
16      return part
```

Spiegazione algoritmo 1.6.1: Spiegazione dell'algoritmo di partizione

Il codice sfrutta una visita in profondità (DFS) modificata per verificare se un grafo ammette una partizione (ad esempio se è bipartito).

- L'array `part` tiene traccia dello stato di colorazione di ciascun nodo: 0 indica non visitato/non colorato, mentre 1 e 2 rappresentano le due classi di partizione.
- Per ogni adiacente v del nodo corrente y:
  1. Se v non è stato ancora visitato ('part[v] == 0'), gli viene assegnato il colore opposto rispetto a y tramite la formula $3-\text{part}[y]$, e viene aggiunto allo stack per la visita.
  2. Se v è già stato visitato, si controlla che non abbia lo stesso colore del nodo corrente y. In caso di conflitto (`part[v] == part[y]`), il grafo non può essere partizionato e l'algoritmo termina restituendo una partizione vuota.

1.7 Lavoro su grafi orientati

In un grafo orientato, la visita DFS funziona ugualmente, ma bisogna prestare attenzione all'orientamento degli archi. Tuttavia nel pratico non cambia nulla, in quanto nella lista di adiacenza di un nodo saranno presenti solo i riferimenti ai nodi che può effettivamente raggiungere.

Inoltre, l'**albero delle visite generato dalla DFS su un grafo diretto** si dice **arborescenza**.

Esercizio 1.7.1: Ordinamento topologico di un grafo

Si consideri un processo suddiviso in n sotto-attività (o *task*) $x_1, \ldots, x_n$, soggette a una serie di vincoli di precedenza:

- x_1 prima di x_2
- x_3 prima di x_2
- x_3 prima di x_1
- x_4 prima di x_3

Trovare un ordine di task che rispetti le varie dipendenze fornite in input.

È naturale modellare le attività x_i come i nodi di un grafo diretto. Per quanto riguarda gli archi, stabiliamo la seguente convenzione grafica per rappresentare la relazione di dipendenza temporale:

$$\text{“}x_i \text{ viene prima di } x_j\text{”} \quad \Longleftrightarrow \quad x_i \longleftarrow x_j$$

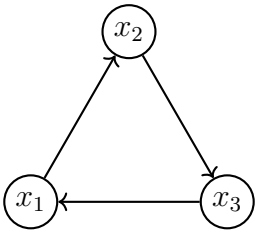
Definizione 1.7.1: Ordine topologico

Risolvere questo problema equivale a individuare un **ordine topologico** dei nodi del grafo, ovvero una permutazione dei suoi vertici $(v_1, v_2, \ldots, v_n)$ tale che, per ogni arco orientato $(u, v) \in E$, il vertice u preceda il vertice v nell'ordinamento.

Chiaramente, una soluzione non esiste sempre. Si consideri ad esempio il seguente insieme di vincoli:

- x_1 prima di x_3
- x_3 prima di x_2
- x_2 prima di x_1

Rappresentando graficamente queste dipendenze si ottiene una struttura ciclica:



Proposizione 1.7.1: Inesistenza dell'ordine topologico

Se il **grafo orientato** G definito dai processi **contiene un ciclo diretto**, allora **non esiste** alcun ordine topologico.

Dimostrazione 1.7.1

Procediamo per assurdo. Assumiamo che esista un ciclo diretto formato dalla sequenza di vertici $(v_1, \ldots, v_k)$ e che, contemporaneamente, esista un ordine topologico valido per il grafo.
Se v_i precede v_{i+1} lungo il ciclo, allora per soddisfare la condizione di chiusura del ciclo deve esistere un arco che dal termine della sequenza ritorna all'inizio. Tuttavia, la presenza dell'arco (v_i, v_{i+1}) rende impossibile preservare l'ordinamento topologico lineare, poiché per definizione tutti gli archi orientati devono puntare coerentemente nella medesima direzione della sequenza (da destra verso sinistra nella notazione stabilita). Di conseguenza, almeno un arco lungo il ciclo violerà necessariamente la relazione d'ordine.

Proposizione 1.7.2: Proprietà dei cicli nei grafi diretti

Se $\nexists$ un ciclo in un grafo diretto $\implies$ $\exists$ un ordine topologico.

Proposizione 1.7.3: Esistenza di vertici senza archi uscenti

Se G è un grafo diretto senza cicli $\implies$ $\exists$ un vertice v senza archi uscenti.

Proposizione 1.7.4: Presenza di cicli nei grafi diretti

Per ogni vertice v, se $\exists$ almeno un arco uscente da v $\implies$ G contiene un ciclo.

Definizione 1.7.2: DAG (Directed Acyclic Graph)

Per notazione, indichiamo con **DAG** (*Directed Acyclic Graph*) un grafo diretto privo di cicli.

1.7.1 Ricerca di un ordine topologico

Vogliamo definire un algoritmo per trovare un **ordine topologico** in un grafo orientato aciclico (DAG).

Osservazione 1.7.1: Proprietà fondamentale

Per la proposizione precedente, sappiamo che se in un grafo orientato esiste un ordine topologico, allora esiste **almeno un nodo senza archi uscenti** (ovvero un nodo pozzo).

Da questa osservazione nasce l'idea per un primo algoritmo. L'idea di base consiste nell'eseguire iterativamente le seguenti operazioni:

1. Scorriamo tutti i vertici del grafo e individuiamo un vertice privo di archi uscenti.
2. Lo rimuoviamo dal grafo.
3. Ripetiamo il procedimento finché non sono stati rimossi tutti i vertici.

Formalizziamo questo approccio tramite il seguente pseudo-codice:

```
Ord(G: grafo diretto aciclico):
  L = []
  while len(G) != 0:
    v = trova_nodo_senza_archi_uscenti(G)
    L.append(v)
    cancella_nodo_e_archi(G, v)
  return L
```

Listing 1.7: Algoritmo ingenuo per l'ordine topologico

Spiegazione algoritmo 1.7.1: Ordine topologico (inefficiente)

Analizziamo nel dettaglio le singole istruzioni dell'algoritmo e i relativi costi computazionali:

- **Inizializzazione e ciclo principale**: Il ciclo `while` viene eseguito $\Theta(n)$ volte, poiché a ogni iterazione viene rimosso esattamente un nodo dal grafo.
- **Ricerca del nodo senza figli**: Per individuare un vertice senza archi uscenti (ovvero un nodo la cui lista di adiacenza sia vuota), dobbiamo scorrere i nodi del grafo, con un costo di $O(n)$.
- **Cancellazione del nodo**: La rimozione del nodo v dal grafo e la successiva eliminazione di tutte le sue occorrenze nelle liste di adiacenza degli altri nodi richiede di scandire l'intera struttura dati, con un costo pari a $O(n + m)$.

Il costo totale dell'algoritmo risulta quindi essere:

$$\Theta(n) \cdot O(n+m) = O(n(n+m))$$

Osservazione 1.7.2: Presenza di più nodi pozzo

Anche se nel grafo dovessero essere presenti più nodi contemporaneamente privi di archi uscenti, l'algoritmo funziona correttamente. Esso restituirà comunque un ordine topologico valido (ricordando che per uno stesso DAG possono esistere più ordini topologici distinti).

Come si nota dall'analisi della complessità, si tratta di un metodo **molto costoso** dal punto di vista computazionale, oltre al fatto che modifica in modo distruttivo la struttura del grafo originale.

Esercizio 1.7.2: Trovare un ciclo in un grafo diretto

Dato un grafo diretto G, trovare un ciclo al suo interno.

```
    def trovaCicloDiretto(G):
      L = []  # Per memorizzare i nodi del ciclo
      x = vertice qualsiasi
      L.append(x)

      vis = [0] * n
      vis[x] = 1  # Segniamo il nodo come visitato

      current = x
      next = x.neighbr[0]

      # Iteriamo finche non troviamo un nodo gia visitato
      while vis[next] == 0:
        vis[next] = 1
        L.append(next)

        current = next
        next = current.neighbr[0]  # (1)

      while L[0] != next:
```

```
21          L.remove()
22
23      return L
```

Listing 1.8: Algoritmo per la ricerca di un ciclo diretto

Spiegazione algoritmo 1.7.2: Ricerca di un ciclo diretto

1. **Inizializzazione**: Viene creata una lista L per memorizzare il percorso e un vettore di tracciamento `vis` di dimensione n inizializzato a 0. Si seleziona un vertice di partenza arbitrario x, lo si inserisce in L e si imposta `vis[x] = 1`.

2. **Esplorazione**: Si imposta `current = x` e come vertice successivo `next` il primo adiacente uscente (`x.neighbr[0]`).

3. **Ricerca del ciclo**: Finché il nodo `next` non è stato visitato (`vis[next] == 0`):

   - Si marca `next` come visitato (`vis[next] = 1`).
   - Si aggiunge `next` alla lista L.
   - Si avanza ponendo `current = next` e selezionando il nuovo vicino uscente `next = current.neighbr[0]` (istruzione **(1)**).

4. **Estrazione della sequenza del ciclo**: Quando il ciclo `while` termina, significa che `next` è un nodo già visitato (è stato ri-incontrato). Il secondo ciclo `while` rimuove gli elementi in testa a L finché l'elemento di testa non coincide con `next`, lasciando in L solo ed esclusivamente i nodi che compongono il ciclo diretto trovato.

L'unica sostanziale differenza rispetto al caso non orientato è un'accortezza: nell'istruzione **(1)** possiamo permetterci di assegnare come prossimo nodo sempre il primo dei vicini (`current.neighbr[0]`). Infatti, essendo il grafo **diretto**, anche la presenza di un eventuale doppio arco orientato tra due soli nodi costituisce a tutti gli effetti un ciclo valido.

Esercizio 1.7.3: Problema dell'antenato comune più vicino

Dato P il vettore dei padri di un albero e dati due vertici x ed y, l'obiettivo è trovare l'antenato comune più vicino (noto anche come Lowest Common Ancestor o LCA)

tra x e y.

Presentiamo una soluzione basata sull'utilizzo di liste puntate per risalire la gerarchia dei padri fino alla radice.

```
def sol1(P, x, y):
  L1, L2 liste puntate
  Do:
    L1.push(x)
    x = P[x]
  while x != P[x]

  Do:
    L2.push(y)
    y = P[y]
  while y != P[y]

  Current = L1[0]
  while L1[1] == L2[1]:
    Current = L1[1]
    L1.remove()
    L2.remove()

  return Current
```

Osservazione 1.7.3

Essendo `L1` ed `L2` liste puntate, gli elementi vengono aggiunti in testa durante la risalita verso la radice. Di conseguenza, alla fine dell'inserimento, il primo elemento della lista corrisponde proprio alla radice dell'albero, mentre gli ultimi elementi rappresentano i nodi di partenza x ed y.

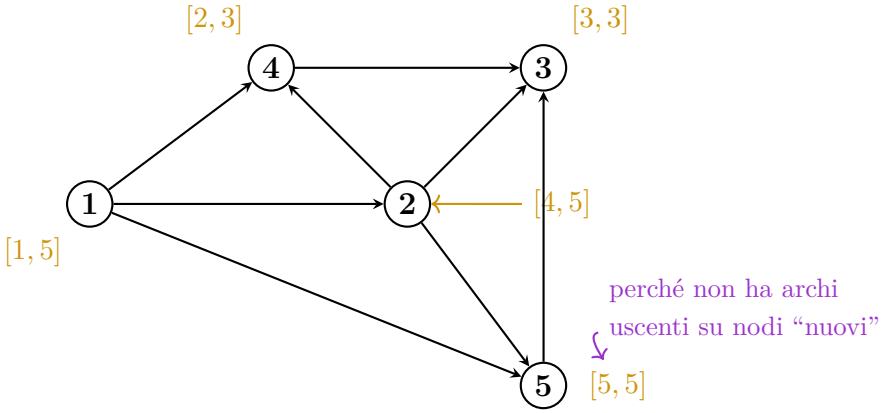
Tornando agli ordini topologici... In precedenza era stata presentata una versione inefficiente con complessità temporale pari a $O(n(n+m))$. È possibile progettare una versione decisamente più efficiente sfruttando una visita in profondità (**DFS**).

Prima di definire formalmente l'algoritmo, introduciamo alcune nozioni teoriche fondamentali relative alla gestione del tempo durante la visita.

Definizione 1.7.3: Tempi di scoperta e fine visita in DFS

Dato un grafo G e una visita DFS su di esso, introduciamo un **contatore globale che viene incrementato ad ogni nuovo nodo visitato**. Ad ogni nodo v viene associata una coppia di valori temporali:

- **Tempo di scoperta** $t(v)$: indica il valore del contatore nel momento in cui il nodo v viene inserito nello stack della DFS. Sostanzialmente, rappresenta l'istante in cui il vertice viene visitato per la prima volta.
- **Tempo di fine visita** $T(v)$: indica il valore del contatore nel momento in cui il nodo v viene rimosso dallo stack della DFS. Sostanzialmente, indica quanti nodi sono stati esplorati complessivamente prima di eliminare v dallo stack (ovvero quando l'esplorazione di v e di tutti i suoi discendenti è terminata).



Consideriamo il grafo sopra rappresentato ed analizziamo l'evoluzione della visita in profondità (DFS):

1. **Partenza dal nodo radice**: supponiamo di partire dal nodo 1. Poniamo $t(1) = 1$, in quanto è il primo vertice visitato.
2. **Attraversamento di un arco uscente**: scegliamo un arco uscente arbitrario, ad esempio verso il nodo 4. Poniamo $t(4) = 2$, essendo il secondo vertice visitato.
3. **Avanzamento**: ci muoviamo verso l'unico nodo adiacente disponibile, il nodo 3, e poniamo $t(3) = 3$.
4. **Backtracking da nodo senza uscite non visitate**: poiché il nodo 3 non possiede archi uscenti verso nodi ancora da scoprire, lo rimuoviamo dallo stack per effettuare il backtracking. Poniamo quindi $T(3) = 3$. Si noti che si tratta dello stesso contatore, ma in questo passaggio non viene incrementato.

5. **Ritorno al predecessore**: torniamo al nodo 4; dato che non ha altri archi uscenti verso nodi non ancora visitati, lo eliminiamo dallo stack e assegniamo $T(4) = 3$.

6. **Prosecuzione**: si procede in modo analogo per i restanti vertici del grafo.

Da questo meccanismo possiamo ricavare due proprietà fondamentali sui tempi di scoperta e di fine visita:

Osservazione 1.7.4: Tempi della radice

Se il grafo è connesso e v è la radice dell'albero di visita, si ha necessariamente:

$$t(v) = 1, \qquad T(v) = n$$

Ciò è dovuto al fatto che la radice è il primo nodo ad essere scoperto ed è l'ultimo ad essere rimosso dallo stack.

Osservazione 1.7.5: Relazione tra tempo di inizio e fine visita

Per ogni vertice $v \in V$ vale la disuguaglianza:

$$t(v) \leq T(v)$$

in quanto un nodo deve essere necessariamente scoperto prima di poter terminare la sua elaborazione ed essere rimosso.

Dunque, ad ogni vertice v risulta associato un intervallo temporale ben definito:

$$[t(v), T(v)]$$

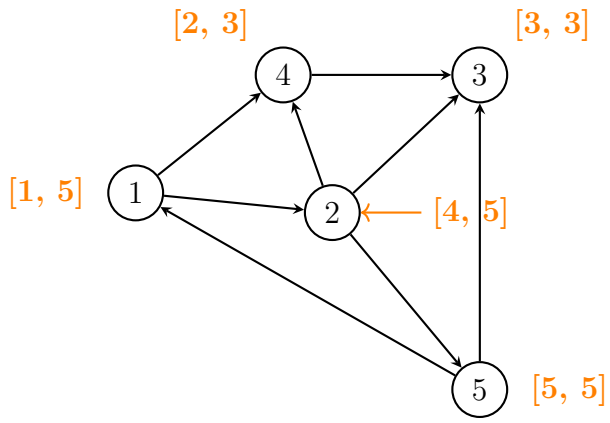
Proposizione 1.7.5: Classificazione degli archi in una DFS

Per ogni coppia di vertici u, v, con u la coda dell'arco e v la testa, vale solo una di queste relazioni:

i) $[t(u), T(u)] \subseteq [t(v), T(v)]$: **ARCO ALL'INDIETRO**

ii) $[t(v), T(v)] \subseteq [t(u), T(u)]$: **ARCO IN AVANTI**

iii) $[t(u), T(u)] \cap [t(v), T(v)] = \emptyset$: **ARCO DI ATTRAVERSAMENTO**



In questo esempio abbiamo:

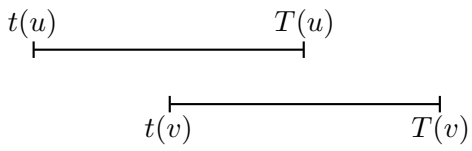
- $(1, 4)$ = **AVANTI**
- $(4, 3)$ = **AVANTI**
- $(1, 2)$ = **AVANTI**
- $(2, 3)$ = **ATTRAVERSAMENTO**
- $(2, 5)$ = **AVANTI**
- $(5, 3)$ = **ATTRAVERSAMENTO**
- $(5, 1)$ = **INDIETRO**
- $(2, 4)$ = **ATTRAVERSAMENTO**

Ragionando tenendo conto dell'arborescenza, cioè dell'albero delle visite che si crea dalla successione dei nodi che appaiono in una DFS su un grafo diretto, vediamo come:

- **ARCHI IN AVANTI:** archi che vanno da un nodo ad un suo **discendente**.
- **ARCHI ALL'INDIETRO:** archi che vanno da un nodo ad un suo **antenato**.
- **ARCHI DI ATTRAVERSAMENTO:** archi che vanno da un nodo ad un suo **"cugino"**.

Dimostrazione 1.7.2

Per dimostrare che vale almeno una di queste proprietà, è necessario dimostrare che **non esiste** alcun caso in cui i due intervalli si sovrappongono parzialmente.



Cioè, una situazione in cui $t(u) < t(v) \leq T(u) < T(v)$ è **impossibile**.
Supponiamo che u sia il nodo inserito per primo nello stack. Di conseguenza, deve valere necessariamente $t(u) < t(v)$, dato che v viene visitato successivamente.

Per la natura LIFO (Last In, First Out) dello stack, il nodo v deve essere rimosso prima di u, poiché l'ultimo elemento a essere visitato deve essere anche il primo a essere rimosso. Siccome v viene sicuramente rimosso prima di u, è **impossibile** che si verifichi $T(v) > T(u)$.

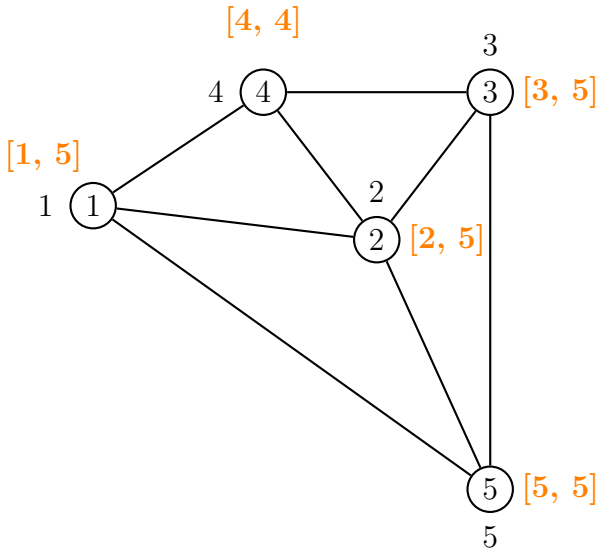
1.8 Comportamento nei grafi non diretti

Come si comporta questa implementazione nei **grafi non diretti**?

Siccome il grafo non è orientato, si possono verificare solo due possibili casi per gli intervalli di visita di qualsiasi coppia di nodi u, v:

i/ii) $I_u \subseteq I_v$ oppure $I_v \subseteq I_u$

iii) $I_u \cap I_v = \emptyset$



Nota: possiamo scegliere noi un ordine di visita dei nodi, di conseguenza l'intervallo segnato nel grafo è riportato solo a scopo esemplificativo.

Proposizione 1.8.1

Dato un DFS su un grafo **non diretto**, se un arco (u, v) non fa parte dell'**albero di visita** (ovvero è un arco all'indietro/back edge), allora è impossibile che si verifichi il caso **iii)** degli intervalli disgiunti.

Dimostrazione 1.8.1

Supponiamo per assurdo che gli intervalli siano disgiunti, ovvero che esista un arco (u, v) tale che:

$$I_u \cap I_v = \emptyset$$

Sicuramente uno dei due nodi verrà visitato prima dell'altro; senza perdita di generalità, assumiamo che questo sia u, ovvero che $t(u) < t(v)$.
Data questa assunzione e data la natura LIFO dello stack, è impossibile che u venga rimosso dallo stack prima che inizi la visita di v, poiché l'esistenza dell'arco (u, v) impone che v venga scoperto prima della chiusura di u. Di conseguenza, deve valere:

$$t(v) \leq T(u)$$

Il che dimostra che gli intervalli non sono disgiunti, giungendo a una contraddizione.

Osservazione 1.8.1: Archi fuori dall'albero in grafi non orientati

Tutti gli archi (u, v) che non fanno parte dell'albero di visita di una DFS su un grafo non orientato sono necessariamente **archi all'indietro** (o in avanti; non essendoci un orientamento nel grafo, la distinzione non è rilevante).

Proposizione 1.8.2: Caratterizzazione dei cicli in grafi non orientati

Sia G un grafo non orientato connesso.

$$G \text{ ha un ciclo} \iff \text{ogni DFS ha almeno un arco all'indietro}$$

Dimostrazione 1.8.2

Analizziamo le due direzioni della doppia implicazione:

- **Direzione** " $\Longrightarrow$ ":
  Abbiamo osservato in precedenza che tutti gli archi che non appartengono all'albero di visita DFS sono archi all'indietro. Dato un ciclo C nel grafo, ed essendo l'albero di visita un grafo aciclico connesso per definizione, deve necessariamente esistere almeno un arco di C che non appartiene all'albero. Tale arco sarà quindi un arco all'indietro.

- **Direzione** " $\Longleftarrow$ ":
  Sia (u, v) un arco all'indietro, con u discendente di v nell'albero DFS. Per

definizione di discendenza, nell'albero di visita esiste un cammino P da v a u. Di conseguenza, l'unione del cammino P con l'arco all'indietro (u, v) forma un ciclo nel grafo.

Proposizione 1.8.3: Caratterizzazione dei cicli in grafi orientati

Dato un grafo orientato G:

$\exists$ un ciclo in $G \iff \exists$ un arco all'indietro per qualsiasi DFS nel grafo

Dimostrazione 1.8.3

Analizziamo le due direzioni della doppia implicazione:

- **Direzione** " $\Longleftarrow$ ":
  Supponiamo che esista un arco all'indietro (u, v), il che implica che u è un discendente di v nell'albero DFS. Nell'albero delle visite esiste quindi un cammino diretto $v \to u$. L'unione di tale cammino con l'arco diretto (u, v) costituisce un ciclo diretto nel grafo.

- **Direzione** " $\Longrightarrow$ ":
  Sia C un ciclo nel grafo G e sia v il primo vertice di C ad essere visitato dalla DFS. Di conseguenza, tutti gli altri vertici del ciclo C verranno scoperti mentre v è ancora attivo, diventando quindi discendenti di v nell'albero DFS.

  Il vertice v_k che completa il ciclo e si ricongiunge con v darà origine a un arco (v_k, v) che si configura come un arco all'indietro.

  Questo comportamento è garantito per tutti i nodi u del ciclo visitati dopo v, in quanto, trovandosi all'interno del ciclo ed essendo stati scoperti dopo v, verranno terminati (e quindi rimossi dallo stack di ricorsione) prima di v. In termini di tempi di scoperta t e fine visita T:

$$t(v) < t(u) \land T(v) > T(u) \quad \forall u \in C \setminus \{v\}$$

Torniamo molto a monte: vogliamo usare tutte queste supposizioni per dare un codice per trovare l'ordine topologico in un tempo $O(n + m)$ usando la DFS.

Programmiamo ora una DFS ricorsiva che ci permette di ottenere $t[x]$ e $T[x]$ per ogni vertice.

```
    def DFS_ric(G, x, t, T, c):
      for y in x.neighbor:
        if t[y] == -1:
          c += 1
          t[y] = c
          DFS_ric(G, y, t, T, c)
      T[x] = c

    def DFS(G, x):
      t = [-1] * n
      T = [-1] * n
      c = 1
      t[x] = c
      DFS_ric(G, x, t, T, c)
```

Spiegazione algoritmo 1.8.1: Ricerca in profondità (DFS) con tempi di visita

La seguente procedura implementa la visita DFS tenendo traccia dei tempi di scoperta $t[x]$ e di fine visita $T[x]$ per ciascun vertice x.

Spiegazione dettagliata delle istruzioni:

- **Controllo della visita (`t[y] == -1`):** Usiamo l'array t per controllare se abbiamo già visitato il vertice y. Poiché $t[y]$ indica il tempo in cui abbiamo scoperto y, un valore pari a -1 indica che il nodo non è ancora stato visitato.
- **Fine visita del sottoalbero (`T[x] = c`):** Una volta che abbiamo visitato tutto il sottoalbero di x, dobbiamo rimuoverlo dallo stack e andare avanti, dunque valorizziamo il tempo di fine visita $T[x]$ con il valore corrente del contatore c (questo avviene in quanto non stiamo usando uno stack esplicito, bensì lo stack della ricorsione delle chiamate di funzione: la risalita dello stack della ricorsione corrisponde alla rimozione graduale dei nodi dallo stack).
- **Inizializzazione:** Nella funzione principale `DFS(G, x)` inizializziamo i due array t e T di dimensione n con il valore -1, impostiamo il contatore iniziale $c = 1$, marchiamo il nodo di partenza x e infine avviamo la procedura ricorsiva.

Per l'ordine topologico sappiamo che non ci sono archi all'indietro, essendo il grafo aciclico (DAG).

Di conseguenza, per qualsiasi arco (u, v), vale esclusivamente una delle seguenti relazioni tra gli intervalli temporali di visita:

- **Disgiunzione:**
$$[t[v], T[v]] \cap [t[u], T[u]] = \emptyset$$
- **Inclusione:**
$$[t[v], T[v]] \subseteq [t[u], T[u]]$$
In questo caso, l'arco (u, v) rappresenta un arco in avanti oppure un arco di **arborescenza**.

Dunque, per ogni arco (u, v) del grafo, vale sempre la relazione:

$$T[v] \leq T[u]$$

Per ricostruire un ordinamento topologico, dobbiamo considerare l'array dei tempi di fine visita T in ordine decrescente: infatti, l'ultimo vertice ad essere completato nella visita è il primo nell'ordine topologico.



Con un arco che collega x e y, si ha $T[y] \leq T[x]$ e quindi l'arco è orientato come $x \to y$.

Se invece consideriamo i tempi di fine visita T in ordine crescente, il primo vertice ad essere rimosso risulterà essere l'ultimo nell'ordinamento topologico.



Con un arco che collega x e y, si ha $T[y] \geq T[x]$ e quindi l'arco è orientato come $y \to x$.

Abbiamo dunque tutti gli elementi teorici necessari per definire l'algoritmo di ordinamento topologico basato su DFS.

```
1    def OrdTop(G):
2      L = []
3      vis = [0] * n
4      for v in G:
```

```
        if vis[v] == 0:
          OrdTopRic(G, v, vis, L)
      return L

    def OrdTopRic(G, v, vis, L):
      vis[v] = 1
      for u in v.neighbors:
        if vis[u] == 0:
          OrdTopRic(G, u, vis, L)
      L.append(v)
```

Spiegazione algoritmo 1.8.2: Ordinamento Topologico

- **Inizializzazione (`OrdTop`):** viene predisposta una lista L inizialmente vuota per raccogliere i nodi ordinati e un vettore booleano `vis` di dimensione n inizializzato a 0, impiegato per tracciare lo stato di visita di ciascun vertice.
- **Ciclo principale:** per ogni vertice $v \in G$, se v non è ancora stato visitato (`vis[v] == 0`), si avvia la procedura ricorsiva `OrdTopRic(G, v, vis, L)`. Al termine delle chiamate per tutte le componenti connesse, viene restituita la lista L.
- **Procedura ricorsiva (`OrdTopRic`):**
  1. Il vertice corrente v viene marcato come visitato impostando `vis[v] = 1`.
  2. Si esaminano tutti i vertici u uscenti da v (`v.neighbors`): se un vicino u non è ancora stato visitato (`vis[u] == 0`), la funzione richiama ricorsivamente se stessa su u.
  3. Non appena tutti i discendenti di v sono stati esplorati (visita in post-ordine), il nodo v viene inserito in L tramite l'operazione `L.append(v)`. L'inserimento del vertice v come ultimo elemento della lista è un modo molto furbo di utilizzare la risalita dello stack della ricorsione per inserire per ultimo letteralmente *l'ultimo vertice a cui siamo arrivati*, ovvero quello con la priorità minore (l'ultimo dell'ordine topologico).

La procedura richiama perfettamente una classica operazione di DFS, dunque il costo computazionale sarà $\mathbf{O(n + m)}$.

Definizione 1.8.1: Pozzo universale

In un grafo diretto G, un **pozzo universale** è un vertice $x \in V(G)$ tale che:

$$\forall y \in V(G) \setminus \{x\}, \quad (y, x) \in E(G) \land (x, y) \notin E(G)$$

In termini intuitivi, si tratta di un nodo a cui arrivano archi da tutti gli altri vertici del grafo, ma dal quale non parte alcun arco verso l'esterno.

Esercizio 1.8.1: Ricerca del pozzo universale in tempo

Dato un grafo G con n vertici memorizzato tramite una matrice di adiacenza, l'obiettivo è stabilire se esiste un pozzo universale in tempo lineare $O(n)$.

Idea intuitiva La matrice di adiacenza ci permette di verificare la presenza di un arco tra due nodi i e j in tempo costante $O(1)$. Da un singolo controllo sulla cella $G[i][j]$ possiamo ricavare due informazioni cruciali che ci consentono di escludere uno dei due nodi come potenziale pozzo:

- **Se** $G[i][j] == 0$: significa che non esiste un arco orientato da i a j. Di conseguenza, j **non può** essere un pozzo universale, poiché un pozzo universale deve essere raggiunto da tutti gli altri nodi del grafo (incluso i).
- **Se** $G[i][j] == 1$: significa che esiste un arco orientato da i a j. Di conseguenza, i **non può** essere un pozzo universale, poiché da un pozzo universale non deve partire alcun arco verso l'esterno.

Grazie a questa proprietà, con un singolo confronto siamo in grado di scartare un candidato, riducendo progressivamente lo spazio di ricerca.
Inoltre, per la definizione di pozzo universale, possiamo ricavare un'altra informazione dalla matrice di adiacenza.

- La riga di indice `candidato_pozzo` deve essere interamente composta da 0, in quanto dal pozzo non devono partire archi.
- La colonna di indice `candidato_pozzo` deve essere interamente composta da 1 (fatta eccezione per l'elemento `G[candidato][candidato]`, in quanto elemento della diagonale e dato che non stiamo ammettendo archi riflessivi).

```
1   def trovaPozzo(G):
2     i, j = 1, 2
3     candidato = 1
4     n = len(G)
5
6     while i <= n and j <= n:
7       if G[i][j] == 0:
8         j += 1
9         candidato = i
10      else:
11        i += 1
12        candidato = j
13
14    for k in range(1, n + 1):
15      if candidato == k:
16        continue
17      if G[k][candidato] != 1:
18        return null
19      if G[candidato][k] != 0:
20        return null
21
22    return candidato
```

Spiegazione algoritmo 1.8.3: Ricerca di un pozzo in tempo $O(n)$

1. **Fase di eliminazione (righe 2-13)**:
   - Vengono inizializzati due indici $i = 1$ e $j = 2$ per scorrere la matrice di adiacenza, e una variabile `candidato = 1` per memorizzare il nodo che attualmente potrebbe essere il pozzo.
   - Nel ciclo `while`, finché entrambi gli indici sono entro i limiti della matrice ($i \leq n$ e $j \leq n$), si effettua il controllo sulla cella $G[i][j]$.
   - Se $G[i][j] == 0$, escludiamo j incrementando il suo indice (j++) e impostiamo il nuovo `candidato` a i.
   - Se $G[i][j] == 1$, escludiamo i incrementando il suo indice (i++) e impostiamo il nuovo `candidato` a j.

- Al termine del ciclo, la variabile `candidato` conterrà l'unico nodo superstite che può effettivamente essere un pozzo universale.

2. **Fase di verifica (righe 15-21)**:

   - Poiché la prima fase garantisce solo che `candidato` sia l'unico possibile pozzo (ma non che lo sia effettivamente), è necessario effettuare una verifica finale in tempo $O(n)$.
   - Si scorre ogni nodo k del grafo (escludendo il candidato stesso):
     - Si controlla che esista l'arco da k al candidato ($G[k][\texttt{candidato}] == 1$). Se manca anche solo un arco entrante, l'algoritmo fallisce ritornando `null`.
     - Si controlla che non esista alcun arco dal candidato a k ($G[\texttt{candidato}][k] == 0$). Se viene trovato un arco uscente, l'algoritmo fallisce ritornando `null`.
   - Se tutti i controlli hanno esito positivo, il `candidato` è confermato come pozzo universale e viene restituito (riga 23).